

SafeMask · Records 反馈式脱敏精度提升方案

XiaoSheng

2026-07-18

SafeMask · Records 反馈式脱敏精度提升方案

本文档罗列基于 `<exe_dir>/records/{YYYY}/*.md` 历史脱敏映射，反哺 SafeMask 引擎（模型调优 / RAG）以提升脱敏精度的所有候选方案，供审查后决定实施路线。

请求复述

- **目标**：将 records 文件夹沉淀的历史数据用于模型精度提升
- **形式**：模型微调 或 RAG 检索增强
- **要求**：所有方案必须符合**第一性原理**、**高质量**、**可审查**

第一性原理约束

1. **100% 离线**——SafeMask 核心承诺，方案不得引入云 API
2. **单用户数据体量小**——通常百到几千条记录，从零训练不现实
3. **记录不是人工标注**——`masked` 字段是当前引擎自己的输出；未经人工反馈，直接训练只会强化模型现有偏见（garbage in → garbage out）
4. **PII 敏感**——记录含明文原文，任何管线必须防泄漏
5. **假阴性严重于假阳性**——漏脱敏泄漏 PII，过脱敏只是打扰

现状读数

- Records schema (v1) : 原始 / 脱敏后 / 识别实体(type, start, end, label) / 模式 (SHADOW|SENTRY) / 时间戳
 - 缺失字段: 置信度、识别器来源、模型版本、人工反馈
 - Engine 现状: AC 字典 + Regex + ONNX NER + Context Enhancer + Checksum 五识别器混合
-

Tier A — 规则挖掘 (低成本 · 立竿见影)

A1. 从命中实体反挖模式 → 扩充 RegexRecognizer

- 对同类实体 (PHONE / EMAIL / ...) 聚类, 用广义后缀树或 differential-pattern-mining 提炼未覆盖的变体
- 例: records 频繁出现 +86 138-XXXX-XX 而现有规则漏了带空格写法 → 自动生成候选正则
- 产出: custom/mined_rules.yaml, 人工审核后合并
- 优势: 零 ML 依赖, 即时收益, 可解释
- 劣势: 对开放式实体 (PERSON、ADDRESS) 无能为力

A2. Aho-Corasick 字典热更新

- 对被脱敏 $\geq N$ 次的具名实体 (人名 / 公司名) 自动加入 AC 词典
- 用 IDF 阈值过滤通用词, 避免 “张先生” 之类高频称谓也被词典化
- 产出: 增量字典文件 + 引擎热重载
- 风险: PII 反向沉淀到字典 → 需加密存储 + 只在本地使用

A3. Context Trigger 挖掘

- 从 original 里抽取实体前后 ± 5 token 的窗口, 聚合触发词 (联系人: / 账号: / SN: / ...)
 - 灌入 context_enhancer.rs 的触发规则表
 - 优势: 直接强化现有 Context Enhancer 层
-

Tier B — NER 模型微调（中成本 · 高上限）

B1. Self-Training / Pseudo-Label 弱监督

- 用当前引擎的高置信度输出作为标签，训练改进版
- **必须**做置信度过滤 + 一致性正则化（consistency training），否则只是过拟合当前错误
- **前提**：`model_q4.onnx` 反向找到 PyTorch checkpoint（`openai/privacy-filter` 在 HuggingFace 开源）
- **产出**：新的量化 ONNX，替换 `src-tauri/models/privacy-filter/`

B2. Human-in-the-Loop 微调（推荐）

- 在 History 页加“标注模式”：✓ 正确 / ✗ 漏脱敏 / ✖ 错脱敏 + 修正
- 累积 ≥500 条人工标签后触发本地训练
- **训练执行方式**（都要本地）：
 - PyTorch + transformers 训练脚本，跑在 CPU / CUDA
 - 或走 `cargo run --bin train`（引入 `candle-rs` 或 `burn` 作为纯 Rust 训练栈）
- **产出**：用户自有微调版模型
- **风险**：训练依赖重（PyTorch / CUDA），可能需独立 sidecar

B3. LoRA / PEFT 适配器

- 冻结基模型，只训练低秩适配器（几 MB）
- ONNX Runtime 目前对动态 LoRA 支持有限 → 训完把 LoRA merge 回权重再导出 ONNX
- **优势**：数据量小时（100–500 条）足够；适配器可版本化、可回滚
- **劣势**：依然需要 PyTorch 训练环境

B4. Domain-Adaptive Pretraining (DAPT)

- 先在用户领域文本（脱敏前的 raw text）上继续 MLM 预训练，再做 NER 微调
- **前提**：用户领域显著偏离原模型分布（金融 / 医疗 / 代码）时收益大
- **劣势**：数据需求量最大，单用户不够；多用户会破坏离线承诺

B5. Continual Learning + EWC

- 弹性权重巩固防止灾难性遗忘
- 适合“每周增量再训”场景

- **复杂度高**，建议做完 B2 再考虑
-

Tier C — RAG / 检索增强（中成本 • 与模型解耦）

C1. kNN Decision Calibrator（推荐 • 第一性原理清晰）

- 把历史决策向量化（`entity_span` 上下文 → `embedding`），存本地 vector store（`hnsw-rs` 或 嵌入式 `qdrant`）
- 推理时：候选 span 检索 k 近邻
 - 邻居多为“确认 PII” → boost 分数
 - 邻居多为“用户回撤” → suppress
- **关键区别**：不改模型权重，只做**决策校准**，完全可逆、可解释
- **产出**：
 - 新增 `core/recognizer/knn_calibrator.rs`，作为一个新识别器在 registry context 阶段之后运行
 - 用 `fastembed-rs`（本地 CPU 小模型，如 `bge-small-zh-onnx`，~50 MB）出向量
- **优势**：冷启动零门槛；数据越多越准；不需重新训练
- **劣势**：需一个额外的小型 embedding 模型；vector store 加密存储

C2. Prototype Bank

- C1 的极简版：不存全部 span，只存每类实体的“原型向量”（K-means centroids）
- 内存占用极小（每类 100 KB 级）
- 精度不如 C1，但工程复杂度低

C3. 本地 LLM 二次校验

- Qwen2.5-0.5B / Phi-3.5-mini INT4（~400 MB）作为“仲裁员”，对引擎标为“边界”的候选做二次判定
 - 完全本地，走 `ort` 推理
 - **优势**：语义理解强，能处理罕见实体
 - **劣势**：延迟大（100–500 ms）；需另建下载/加载管线；模型质量参差
 - **建议**：作为可选功能；SHADOW 模式下用户主动粘贴时才推理；不在 SENTRY 每次剪贴板变动都跑
-

Tier D — 反馈闭环（工程投入 · 数据质量的地基）

没有 D, B / C 都是空中楼阁。records 目前没有 ground truth, 直接拿去训练只会强化现有偏见。

D1. History 页标注 UI

- 每条记录 3 个动作: ✓ / ✗ / ✎ 修正
- record markdown 增字段 `feedback: correct | missed | over_masked | corrected`

D2. 隐式反馈信号

- 用户复制 → SafeMask 脱敏 → 用户粘贴 → **手工把某些位置改回明文** (说明过脱敏)
 - 需要监听剪贴板 + 目标应用 (隐私风险高, 谨慎)
- 或: Magic Paste 之后用户立即复制修改版 → 学习 diff

D3. Uncertainty Sampling

- Engine 每 span 输出置信度 (NER 已有 logits), 把低置信度 span 优先弹给用户标
- 是 B2 训练效率的关键杠杆

D4. Records Schema v2

- 新增字段: `confidence`, `recognizer_source`, `model_version`, `user_feedback`
 - 老记录标 `schema_version: 1` 保持兼容
-

Tier E — 对抗式验证（守门员）

E1. False Positive Discriminator

- 训练轻量二分类器 (是不是真 PII), 只跑在主 NER 判定为实体的 span 上
- 输入极少, 可用 `linfa-logistic` 纯 Rust 实现
- **杠杆点**: 假阳性率↓ → 用户反馈成本↓ → B2 数据质量↑

E2. Ensemble Voting

- 主 NER + 规则 + kNN 三票制，任意两票命中即输出
 - 权重从 records 拟合（少量标注样本学最优阈值）
-

Tier F — 数据质量与安全（必做前置）

F1. Records 加密存储

- 记录含明文 PII，磁盘泄漏风险大 → age / AES-GCM + 设备指纹派生密钥
- 复用现有 `download_auth.rs` 的 HMAC 机制

F2. 去重与近邻聚合

- 用 SimHash 去除重复复制的同内容，防止训练集失衡

F3. 数据卡 (Datasheet)

- 每次训练前统计：实体分布、样本数、时间跨度、假阳性密度
- 触发训练的最小阈值卡（如 <500 条不训）

F4. 隐私预算

- 若将来做多用户联邦学习，需 DP-SGD
 - 当前单用户可跳过
-

推荐路线（按第一性原理 × 投产比排序）

Phase 1（当前季度）

- **F1 + F4**: records 加密 & 版本化 → 数据地基
- **D1 + D4**: 反馈 UI + schema v2 → 建立 ground truth
- **A1 + A2**: 规则 / 字典挖掘 → 立竿见影

Phase 2 (累积 500+ 标签后)

- **C1**: kNN Calibrator → 与模型解耦的精度杠杆
- **E1**: 假阳性判别器 → 收窄错误面

Phase 3 (稳定人工反馈后)

- **B2 或 B3** (LoRA) → 真正的模型自适应

长期观察

- **C3**: 本地 LLM 仲裁 → 视 Qwen / Phi 4-bit 推理性能演进
- **B4**: DAPT → 若未来接入多用户匿名语料

关键决策点 (需审查)

1. 是否接受“人工标注”作为前置条件？ 若不接受，Tier B 全部作废，只能走 A + C 的启发式路线
2. 是否允许将来引入 PyTorch / candle 作为训练侧依赖？ 若不允许，B / E 的模型训练部分需重新设计（训练可在独立 CLI，运行时仍纯 ONNX）
3. 是否愿意增加 embedding 模型 (~50 MB) 作为 C1 前置？ 决定 RAG 路线可行性
4. records 是否允许迁移到加密格式？ 决定 F1 推行速度（会破坏现有 Markdown 可读性 → 也可保持 md 明文 + 单独加密 index）

附录 A — 目前 Records 数据结构

```
## 记录 N

### 原始内容
```

{original text}

脱敏后内容

{masked text}

识别实体

类型	起始	结束	脱敏值
PHONE	6	17	138****8000

- ### 统计
- 模式: SHADOW
 - 实体数: 1
 - 时间: 14:30:00

附录 B — 相关文件锚点

- 记录写入: `src-tauri/src/infra/record_writer/markdown.rs`
- 引擎主循环: `src-tauri/src/core/hybrid_engine.rs`
- 识别器注册: `src-tauri/src/core/recognizer/registry.rs`
- 上下文增强: `src-tauri/src/core/recognizer/context_enhancer.rs`
- 配置: `src-tauri/src/core/config.rs` (`record_writer_enabled` 等)